

# **Containerization — handout**

---

# Contents

---

## 1 Opening — the thesis, the plan and the scope (slides 1 to 5)

1.1 What this part shows

1.2 How it was shown

1.3 Why it matters

## 2 Part I — Why containers exist: the need (slides 6 to 11)

2.1 What this part shows

2.2 How it was shown

2.3 Why it matters

2.4 Terms

## 3 Part II — Run your first container (slides 12 to 43)

3.1 What this part shows

3.2 How it was shown

3.3 Why it matters

3.4 Try it yourself

3.5 Terms

## 4 Part III — What is underneath: a container is a process (slides 44 to 62)

4.1 What this part shows

4.2 How it was shown

4.3 Why it matters

4.4 Try it yourself

4.5 Terms

## 5 Part IV — Two architectures, one idea (slides 63 to 78)

5.1 What this part shows

5.2 How it was shown

5.3 Why it matters

5.4 Try it yourself

5.5 Terms

## 6 Part V — What follows from the architecture (slides 79 to 83)

6.1 What this part shows

6.2 How it was shown

6.3 Why it matters

6.4 Try it yourself

## 7 Part VI — Where the abstraction breaks (slides 84 to 96)

7.1 What this part shows

7.2 How it was shown

7.3 Why it matters

7.4 Try it yourself

8 Part VII — Where it came from, optional (slides 97 to 101)

8.1 What this part shows

8.2 How it was shown

8.3 Why it matters

9 Close — the takeaways and the habits (slides 102 and 103)

9.1 What this part shows

9.2 How it was shown

9.3 Why it matters

*Generated by the agentic vault — an autonomous research and authoring harness. Every factual claim was gated against a cited source registry before this document was produced.*

v3 · dff05f3a8 · built 2026-09-30

**Reading this document:** [S<n>] and [V<n>] cite the numbered sources in the references document. [inference] marks a claim that follows from those sources but is not stated by them. [speculative] marks the author's judgement, offered as judgement.

# 1 Opening — the thesis, the plan and the scope (slides 1 to 5)

---

## 1.1 What this part shows

---

- A container is an ordinary process (slide 1). Unrelated kernel interfaces restrict what it sees and what it consumes [V1]. The host kernel is shared, so the separation is weaker than a virtual machine's separation [V7].
- Two commands differ by one option, and only the second one works (slide 2). A tracer attaches to a running process only with `CAP_SYS_PTRACE` [S67]. Neither engine grants that capability by default [S42] [S43].
- The lecture covers the five kernel mechanisms, running one engine on your own laptop, the two process architectures, and the places the abstraction leaks (slide 5).
- Orchestration across machines, networking beyond one published port, image hardening and supply-chain practice stay out of scope (slide 5).

## 1.2 How it was shown

---

- The roadmap fixes the order of the seven parts (slide 3). You run containers in Part II and see how they work in Part III.
- Five takeaways open the lecture (slide 4) and return in the same words at the end (slide 102).
- Every runnable command appears as a `docker` column and a `podman` column (slide 2). Either engine serves the whole topic, and neither one is the default.

## 1.3 Why it matters

---

- The refusal on slide 2 is the capability machinery working as documented [V10]. A refused command names the mechanism that refused it, and Part III reads that name.
- Each mechanism in Part III answers a question that running the commands raised in Part II (slide 3). The order carries the argument.

## 2 Part I — Why containers exist: the need (slides 6 to 11)

---

### 2.1 What this part shows

---

- One server carries several workloads, and three conditions must hold together (slide 7) [S58]. No workload reads another workload's data, changes how another one runs, or takes so much of the machine that the others stop.
- Two applications on one host can need different versions of the same thing (slide 8) [S58]. The second need is a package that carries the whole user-space environment [inference].
- A virtual machine answers both needs, and it charges for a second kernel (slide 9) [S63]. A microVM keeps that kernel and drops almost every simulated device (slide 10) [S38]. A container keeps neither cost, because it uses the host kernel (slide 11) [V1].

### 2.2 How it was shown

---

- The `chroot` call shows a process one subtree of the file tree and hides the rest (slide 7) [S19]. That compartment does not extend to the process or networking spaces [S19]. The author of FreeBSD jails states that `chroot` was never intended as a security mechanism [S20].
- The Solaris Zones paper records two applications at one customer site, both hard-coded to port 80 (slide 8) [S58]. The customer answered by buying two servers [S58]. Boettiger cites a University of Arizona study where "less than 50% of software could even be successfully built or installed" [S62].

- Felter's paper cites 11 seconds to start a virtual machine, against under 1 second for a container on the same hardware (slide 9) [S63]. Firecracker "boots to application code in less than 125ms" and "offers memory overhead of less than 5MB per container" (slide 10) [S38].

## 2.3 Why it matters

---

- A container isolates workloads that cooperate, and it is one defence among several (slide 7) [V7]. Kubernetes states that containers "offer a weaker isolation boundary than virtual machines that utilize hardware-based virtualization" [S73].
- Start-up time and density are the argument, and raw speed is not (slide 9). Felter's own results put containers "equal or better" on almost every workload, and both forms needed tuning for input-heavy work [S63].
- The three shapes on slide 10 differ in where the boundary sits. A container draws it on the system-call interface of the host kernel, and both machine shapes draw it on hardware plus the monitor [S38].

## 2.4 Terms

---

- **Virtual machine** — a simulated computer that runs its own kernel, so the guest kernel is not the host kernel (slide 9). It boots, and it holds its own memory.
- **MicroVM** — a virtual machine with almost every simulated device removed (slide 10). It keeps its own kernel and its hardware boundary, and gives up the rest.
- **Container** — a process, or a tree of processes, that runs with its own view of the machine on the host kernel (slide 11).

# 3 Part II — Run your first container (slides 12 to 43)

---

## 3.1 What this part shows

---

- A container engine resolves an image name, unpacks the layers, writes a runtime configuration, and starts the process (slide 13) [S10] [S7].
- An image is inert data, and a container is a process started from it (slides 20 and 21) [S8] [V1].
- That process sits in the host process table, and the list it reads itself holds one or two entries (slides 24 and 25) [S18].

- One write has three possible destinations, and the three have three different life-times (slide 33) [S64].
- A memory limit holds, and the report inside the container ignores it (slide 40) [S61] [S41].
- One capability decides whether a tracer attaches to a running process (slide 42) [S67].

## 3.2 How it was shown

---

- Linux installs an engine directly (slide 14). macOS installs Homebrew, then Lima (slides 15 and 16) [S83] [S53]. Windows installs WSL 2 (slide 17) [S46]. Both of those platforms supply a Linux kernel inside a virtual machine (slide 18) [V9].
- Every observation follows a written prediction (slides 23, 26, 39 and 41) [S57].
- A stop and a start keep the file the container wrote, and a remove loses it (slides 26 and 27) [S64]. The image never held that file, and it never changes (slide 28) [S8] [inference].
- A container started with `--memory 256m` still prints the whole machine's memory from `free -h`, and `nproc` still counts the host processors (slides 39 and 40) [S68] [S41]. An allocation of 512 MB under that limit is stopped inside the kernel [S2] [inference].
- Case A traces a child process and works. Case B attaches to a running process and returns `Operation not permitted` (slides 41 and 42) [S67]. The same attach succeeds after `--cap-add SYS_PTRACE` [S67].
- `docker image history` prints one row for each instruction (slide 37) [S60]. The `CMD` row is 0 bytes, because it changed no file [S60].

## 3.3 Why it matters

---

- Part II closes with five open questions and names no mechanism (slide 43). Each kernel mechanism in Part III answers one of them [V1].
- Docker Hub allows an unauthenticated address 100 pulls in six hours (slide 22) [S49]. A room behind one shared address draws on one budget, so pull 101 returns an error that resembles broken Wi-Fi [S49] [inference].
- A bind mount ties the run to one machine, and it exposes the real host files to the container (slide 31) [S80]. On macOS and Windows it also crosses out of the Linux virtual machine, and that crossing is slow [S44] [S72].

- A bind mount can leave files under an owner nobody chose (slide 30). Treat root within a container as root outside of it, because a root daemon writes host files as root [S32] [inference]. Rootless Podman instead maps the account onto the numbers reserved in `/etc/subuid` [S11] [V32].
- A volume carries a name instead of a host path, so the same command runs on any machine (slide 32) [S76].
- A Dockerfile and a Compose file replace the typed history of the run (slides 36 and 38). Compose is the one place where the two engines differ in substance, because Podman ships only a wrapper around an external provider [S66].
- Two host settings decide whether the engine starts at all. Podman 6 needs control groups version 2 (slide 14) [S65]. Rootless Podman under WSL 2 needs the Linux service manager (slide 17) [V38] [S47].

### 3.4 Try it yourself

---

- Open one Linux shell and stay in it for every later command (slide 19). macOS uses `limactl shell docker`, and Windows uses `wsl` [S53] [S46].
- `stat -fc %T /sys/fs/cgroup` prints `cgroup2fs` on a host with control groups version 2 (slide 14).
- `docker run --rm -d --name web -p 8080:80 nginx:alpine`, then `curl -s localhost:8080` (slide 23). Substitute `podman` for `docker` in every command.
- `docker volume create appdata`, then two runs that write and read `/data/f` (slide 32).
- The files `app.py`, the Dockerfile and the Compose file are in the talk's `demo/` folder (slides 35, 36 and 38).
- `docker run --rm --memory 256m alpine sh -c 'free -h; nproc'` (slide 39).

### 3.5 Terms

---

- **Container engine** — the program a user installs and types commands at (slide 13). Docker and Podman are container engines.
- **Image** — a stack of read-only layers plus a manifest that names them by the hash of their content (slide 20). Nothing about it runs.
- **Layer** — one read-only set of filesystem changes (slide 20). A running container adds one writable layer above the stack (slide 28).

- **Registry** — a server that stores images under a name and a content hash, and serves them over a documented protocol (slide 22).
- **Bind mount** — a mount that makes an existing host directory appear at a chosen path inside the container (slide 29).
- **Volume** — a directory the engine creates and tracks under its own name (slide 32). Its lifetime is not the container's lifetime.

## 4 Part III — What is underneath: a container is a process (slides 44 to 62)

---

### 4.1 What this part shows

---

- The container process was in the host process list all along (slide 45) [S18]. A second numbering was added, and inside that numbering the command is process 1.
- A namespace is a kernel object that wraps one global resource (slide 46) [S1]. Linux defines eight types, and each type wraps a different resource (slide 47) [S1].
- Only the user namespace changes which account counts as root, and only it can be created without administrative privilege (slide 49) [S1] [S21].
- A union filesystem presents the read-only layers plus one writable directory as a single tree (slide 50) [S74] [S6]. The first write to a file copies the whole file upwards (slides 51 and 52) [S6].
- A control group limits and measures consumption, and it changes nothing a process reads (slides 54 and 55) [S2].
- A capability is one switchable unit of superuser privilege (slide 57) [S3]. A seccomp filter decides which system calls the process may issue (slide 59) [S4].

### 4.2 How it was shown

---

- `ls -l /proc/self/ns/` prints one symbolic link for each namespace, and the target names the type and a number (slide 46) [S1]. Two processes share a namespace when the two numbers match [S1].
- Three ordinary system calls create or join a namespace (slide 48): `clone(2)`, `unshare(2)` and `setns(2)` [S1]. No special interface exists for the purpose, and a tracer shows all three.

- Creating any namespace type except the user namespace needs `CAP_SYS_ADMIN` (slide 49) [S21]. That single exception is what a container without root rests on [V1].
- The overlay driver copies a 2 GB file when a process appends one byte to it (slide 52) [S6]. It supports 128 read-only layers, and containers reading one file share one page-cache entry [S6].
- Inside a container started with `--memory 256m`, the file `/sys/fs/cgroup/memory.max` holds 268435456, which is 256 MB in bytes (slide 54) [S2].
- Podman's default capability set holds eleven entries, and Docker's holds those eleven plus `CAP_MKNOD`, `CAP_NET_RAW` and `CAP_AUDIT_WRITE` (slide 58) [S42] [S43]. Docker's default seccomp profile refuses about 44 of more than 300 system calls (slide 59) [S5].

## 4.3 Why it matters

---

- The memory report has no kernel fix (slide 56). `/proc/meminfo` and its neighbours always print host numbers, and LXCFS covers those paths with a userspace filesystem [S41] [S69].
- A tracer attaches only with `CAP_SYS_PTRACE`, or to a descendant of itself, and the Yama security module applies that limit (slide 60) [S67] [S84]. Granting the capability also allowlists `ptrace`, `process_vm_readv`, `process_vm_writev` and `kcmp` [S17].
- One configuration file carries all five requests, and every mechanism in this part has a field in it (slide 61) [S7] [S81].
- There is no `struct container`, no call that creates one, and no subsystem that owns the word (slide 62) [V2]. Removing any one of the five still leaves something that runs, so the answerable question is which of the five a command requested [V2].

## 4.4 Try it yourself

---

- `ls -l /proc/self/ns/` in the shell, then the same command inside a container, and compare the numbers (slide 46).
- `sudo unshare --pid --fork --mount-proc ps -ef` prints a two-line process list, and no engine ran (slide 48).
- `docker run --rm -v appdata:/data alpine cat /proc/self/mounts` shows an entry the host table does not have (slide 53).
- `docker run --rm alpine sh -c 'grep Cap /proc/self/status'` prints the capability set the engine left (slide 58).

## 4.5 Terms

---

- **Namespace** — a kernel object that wraps one global system resource (slide 46). The processes inside it see their own instance of that resource.
- **Union filesystem** — a filesystem that presents read-only directories plus one writable directory as a single tree (slide 50). The Linux implementation used here is `overlayfs`.
- **Control group** — a kernel mechanism that organises processes into hierarchical groups whose use of a resource is limited and measured (slide 54).
- **Capability** — one independently switchable unit of the privilege traditionally held by the superuser (slide 57).
- **Seccomp filter** — a small program the kernel runs before each system call (slide 59). It returns allow, an error, a kill, or a notification to a tracer.

## 5 Part IV — Two architectures, one idea (slides 63 to 78)

---

### 5.1 What this part shows

---

- The engine writes the request, and the OCI runtime issues it (slide 64) [V29]. The programs `runc`, `crun` and `youki` implement one published standard, and they are interchangeable [S7] [V16].
- Docker runs a client, a daemon, `containerd`, a shim and the OCI runtime (slide 65) [S10] [S22]. Podman runs one command, `common` and the OCI runtime (slide 68) [S13] [S15].
- The Docker daemon is not the container's parent (slide 66) [S70]. It controls the life-cycle by policy, and live restore is not the default (slide 67) [S12].
- Both engines make the same five requests to the same kernel, and they disagree about which process owns the container (slide 78) [V1] [V15].

### 5.2 How it was shown

---

- A trace of `podman run` holds one `unshare` call that creates five namespaces at once (slides 70 and 71) [S1]. The caller is `runc:[1:CHILD]`, a descendant of the typed command [S15] [inference].

- The same trace of `docker run` holds only a socket connection to the daemon (slide 71) [S10]. An attach to `containerd` shows the same call under the daemon, and that attach needs `sudo` (slide 72) [S70] [S84].
- A start without root runs in a fixed order (slide 69). A pause process holds one user namespace, the identifier map comes from `/etc/subuid`, a mount namespace carries the storage, and a userspace program carries the network [S15] [V32] [V33].
- The `crun` project ran 100 sequential containers in 1.69 seconds, against 3.34 seconds under `runc` on the same benchmark (slide 73) [S16]. A container still started under a 512 KB memory limit with `crun`, and `runc` failed even with 4 MB [S16]. These are the project's own figures.
- One study measured 570 Mbps on a single stream without root, against 17.9 Gbps with root, and attributed the gap to `slirp4netns` (slide 76) [V14].

## 5.3 Why it matters

---

- Rootless means the engine and the container both run as an unprivileged user (slide 74) [S11]. Docker moves its daemon into a user namespace and needs at least 65,536 reserved identifiers [S11]. Podman has no daemon to move [S13].
- Podman states that a rootless container never holds more privilege than the account that launched it [S14]. That sentence is a design statement and not a measurement [inference]. Dirty Pipe escaped a container that was granted no extra privilege [S30].
- Host policy overrides every engine option (slide 75). Ubuntu restricts unprivileged user namespaces by default from release 23.10, and this course's Windows path installs Ubuntu [S77] [S48].
- Podman 6 removed `slirp4netns`, and the shipped default is now `pasta` (slide 76) [S65] [S78].
- One registry serves the same manifest and the same layer archives to both engines (slide 77) [S8]. A second engine is therefore a replacement rather than a copy.

## 5.4 Try it yourself

---

- `ps -ef --forest | grep -A2 containerd-shim` under Docker, and the same listing with `grep -B1 nginx` under Podman (slide 66).
- The two traces on slide 70, as an ordinary account in the Linux shell. Install the tracer first with `sudo apt install strace`.

- `unshare -U true` succeeds where an unprivileged user namespace is permitted (slide 75).

## 5.5 Terms

---

- **OCI runtime** — the program that issues the kernel requests for one container and then runs its command (slide 64). The implementations are interchangeable.
- **Daemon** — a program that keeps running after the command that started it exits (slide 65). It owns resources on the user's behalf.
- **Rootless** — a mode in which the engine and the container both run as an unprivileged user (slide 74). It rests on a user namespace.
- **AppArmor profile** — the confinement rules a Linux security module applies to one named program (slide 75). The `usersns` rule in that profile permits creating a user namespace.

## 6 Part V — What follows from the architecture (slides 79 to 83)

---

### 6.1 What this part shows

---

- Almost every command runs unchanged on the other engine (slide 80) [S13]. Compose is the exception, because Podman ships only a wrapper [S66].
- All five mechanisms are Linux kernel features (slide 81) [S1] [S2] [S3]. The macOS and Windows kernels lack them, so Lima and WSL 2 supply a Linux kernel inside a virtual machine [S53] [S46].
- How many virtual machines an engine builds is a design choice, and the published answers differ (slide 83) [S37].

### 6.2 How it was shown

---

- Microsoft states that WSL 2 uses "a Linux kernel inside of a lightweight utility virtual machine" (slide 81) [S46].
- A Windows container shares the Windows kernel directly, and Microsoft calls that "approximately the same as how Linux containers run" (slide 82) [S35].

- Apple states that its Containerization project "executes each Linux container inside of its own lightweight virtual machine" (slide 83) [S37].

## 6.3 Why it matters

---

- Lima and WSL 2 are instances of the shared-kernel argument, and not work-arounds around it (slide 81) [V9].
- The claim is scoped to Linux containers (slide 82) [inference]. Adding a Linux virtual machine does not help a Windows container.
- Two containers under Apple's shape share no kernel (slide 83) [S37]. The shared-kernel claim therefore describes the usual arrangement and not a law.

## 6.4 Try it yourself

---

- Repeat `run`, `ps`, `build` and `--cap-add SYS_PTRACE` under the other engine (slide 80). Compare the output of each pair.

# 7 Part VI — Where the abstraction breaks (slides 84 to 96)

---

## 7.1 What this part shows

---

- The container boundary is the system-call interface of a shared kernel (slide 85) [V7] [S73]. That interface carries more than 300 calls, and Docker's default profile refuses about 44 of them [S5].
- CVE-2019-5736 overwrote the host OCI runtime binary from inside a container, under default options (slide 86) [S25] [V37]. It reached Docker, Podman, CRI-O and containerd alike [S24].
- One class of `runc` flaw was disclosed four times (slide 87) [V36] [S28]. Each disclosure turned on a path the container itself controlled [inference].
- Dirty Pipe was a kernel flaw instead, and no fix in an OCI runtime could have prevented it (slide 88) [S30].
- A virtual machine's interface is narrower than a system-call interface, and narrower is not absolute (slide 90) [S36].

## 7.2 How it was shown

---

- The escape on slide 86 ran in four steps. A malicious image points its entry at `/proc/self/exe`, and the OCI runtime re-executes itself inside the container [S25]. A library the attacker loaded then opens the host binary for writing [V34]. The next container start runs attacker code as root [V34] [inference].
- November 2025 disclosed CVE-2025-31133, CVE-2025-52565 and CVE-2025-52881, and CVE-2026-41579 followed in 2026 (slide 87) [V36] [S28]. The fixes shipped in runc 1.2.8, 1.3.3 and 1.4.0-rc.3 [S26].
- One survey counted 59 published flaws across the 11 "container runtimes" it examined (slide 89) [S29]. Of the 28 with a working proof of concept, 46 percent enabled an escape [S29].
- User namespaces prevented 7 of the 9 escapes that survey tested (slide 89) [S29]. The two survivors involved the host OCI runtime running as root against container-controlled objects [V35].
- CVE-2025-22225 is a published escape from a VMware ESXi virtual machine (slide 90) [S36]. The comparison is about the width of the interface, and not about a guarantee on either side.
- AWS reported itself unsatisfied with Linux containers for untrusted functions, and built Firecracker instead (slide 91) [S38].

## 7.3 Why it matters

---

- The `runc 1.1.x` branch and everything earlier stayed unpatched (slide 87) [S26]. An untrusted image or build file is the "most likely delivery method" [S27].
- Kata Containers builds a small virtual machine for each container, and the runtime specification carries a virtual-machine section (slide 92) [S33] [S34]. The shared-kernel claim is scoped for that reason.
- On macOS a bind mount crosses out of the virtual machine on every access (slide 93) [V9]. One measurement puts it about 3.5 times slower than Linux with VirtioFS, and about 10 times slower with gRPC-FUSE [S44].
- An image built for another processor architecture fails with `exec format error`, or runs slowly under translation (slide 94) [S59] [S45].
- One scanning study found that images for neuroscience data analysis "contain hundreds of vulnerabilities" (slide 95) [S31]. Rebuilding against current packages removed roughly two thirds of them [S31].
- A namespace can outlive every process inside it, because a bind mount of an entry under `/proc` keeps it alive (slide 96) [S1].

## 7.4 Try it yourself

---

- Read the real memory limit from `/sys/fs/cgroup/memory.max` rather than from `free` (slide 96) [S2] [S41].
- Check `/proc` for a bind mount before assuming that a namespace is gone (slide 96) [S1].
- Read the network backend and the storage driver from `containers.conf` (slide 96) [V27].
- Name the architecture with `--platform linux/arm64` or `--platform linux/amd64` (slide 94) [S61].

## 8 Part VII — Where it came from, optional (slides 97 to 101)

---

### 8.1 What this part shows

---

- One line runs from `chroot` in 1982 to the Open Container Initiative in 2015 (slide 98) [V30] [S9].
- Three lanes met, rather than one line of descent (slide 99) [V5]. Isolation, kernel mechanisms and packaging developed in parallel.
- Isolation was available for a decade before 2013 (slide 100) [V3]. What 2013 added was a layered image and a protocol for moving it between machines [V4].

### 8.2 How it was shown

---

- The 1982 date comes from Kirk McKusick's reading of the source-control logs (slide 98) [V30]. Popular timelines print 1979, and the two accounts genuinely disagree [V31].
- Contemporaneous accounts attribute the idea of namespacing to Plan 9, and not to BSD (slide 99) [V5]. Google's "process containers" were renamed control groups, because the kernel maintainers declined the word [V6].
- Docker began as a command-line front end to LXC and AUFS (slide 100) [S22].
- Kubernetes removed `dockershim` in release 1.24, and images built by Docker kept working (slide 101) [S39] [S40].

## 8.3 Why it matters

---

- The technically interesting part was the least technically novel part (slide 100) [V4].
- A Docker image is an OCI image, so the format outlived control of the implementation that produced it (slide 101) [S40].

# 9 Close — the takeaways and the habits (slides 102 and 103)

---

## 9.1 What this part shows

---

- A container is a list of kernel requests, and not a kernel object (slide 102) [V2].
- The boundary is the system-call interface of a shared kernel, and the record holds real escapes through it (slide 102) [V7].
- The two engines disagree about ownership of the container process, and not about what they ask of the kernel (slide 102) [V15].
- A Linux container on macOS or Windows runs inside a Linux virtual machine (slide 102) [V9].
- What a process may do is a readable capability set and a readable seccomp filter (slide 102) [V10].

## 9.2 How it was shown

---

- The five takeaways opened the lecture on slide 4, and each part delivered one of them.
- The same five takeaways return in the same words at the end (slide 102).
- Every slide therefore traces back to a claim the room can repeat a week later (slide 102).

## 9.3 Why it matters

---

- Four habits follow (slide 103). Read the refusal, choose where the data lives, check the host rather than the option, and keep predicting before observing.

- Three of the four read a value the machine already holds (slide 103): the capability set [S3], the storage destination [S64] and the host's control group version [S65].
- Treat root inside a container as root outside the container (slide 103) [S32]. Never answer a permission error with `--privileged`, which is the absence of a boundary [V8].

**References:** the full list of sources and the summaries of the vault notes are in the references document, `2026-09-24-pb158-containerization-references.pdf`, and on the page `references.html` beside the deck.